

Docker 1

Laboratorium
Bartosz Brudek / Tomasz Goluch
2025/26

1. Wstęp

Zadaniem tego laboratorium jest poznanie podstawowych funkcjonalności Dockera takich jak pobieranie obrazów, uruchamianie ich w kontenerach oraz budowanie obrazów Dockera warstwa po warstwie. Kolejnym krokiem będzie przygotowanie plików *Dockerfile* dla solucji *Library*. Zawiera ona trzy serwisy, które po odpowiedniej konfiguracji powinny się ze sobą komunikować za pomocą różnych protokołów komunikacyjnych. Celem zajęć jest pokazanie, jak w łatwy sposób można zbudować gotowy obraz Dockera przy pomocy plików *Dockerfile*.

2. Przygotowanie środowiska

Aby móc otworzyć i edytować solucję, należy zainstalować:

1. SDK .NET w odpowiedniej wersji (na stronie: <https://dotnet.microsoft.com/en-us/download/dotnet>). Najnowsza wersja biblioteki dla Microsoft Visual 2022.
2. Docker (Oficjalne źródło: <https://www.docker.com/>);

Do tworzenia i konfigurowania plików *Dockerfile* potrzebna będzie wiedza na temat komendy *dotnet publish*. Służy ona do kompilacji projektu i zapakowania plików wynikowych wraz z zależnościami do folderu. Folder ten można potem wgrać na maszynie hosta. Dokładny opis i przykłady można znaleźć na poniższej stronie: <https://learn.microsoft.com/pl-pl/dotnet/core/tools/dotnet-publish>.

3. Stworzenie Dockerfile'a

W systemie opartym na wielu usługach powinno się stosować takie podejście do wdrażania, w którym każda z nich może być uruchamiana oddzielnie. Idealnym narzędziem do tego zadania jest Docker. Każda aplikacja lub usługa powinna posiadać swój plik *Dockerfile*. Dobrą praktyką jest podzielenie go na dwa kroki:

1. pobieranie zależności i budowanie aplikacji na środowisku z narzędziami do budowania;
2. uruchomienie paczki z aplikacją na środowisku uruchomieniowym;

Do stworzenia wszystkich kontenerów podczas laboratorium najlepiej wykorzystać poniższe bazowe obrazy (wyjątkiem jest obraz dla kolejki RabbitMQ, której konfiguracja kontenera zostanie dostarczona wraz z projektem)

- `mcr.microsoft.com/dotnet/sdk:8.0` jako obraz do budowania;
- `mcr.microsoft.com/dotnet/aspnet:8.0` jako obraz do uruchamiania;

Dockerfile, który należy stworzyć dla projektu *Library.Web*, powinien posiadać poniższą strukturę:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build-env
WORKDIR /app

mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app

ENTRYPOINT ["dotnet", "Library.Web.dll"]
```

4. Opis projektów wykorzystanych do laboratorium

Do zadania wykorzystana zostanie gotowa solucja o nazwie Library, w której zawarte są trzy projekty:

- Library.Web – aplikacja webowa z interfejsem graficznym, która wyświetla dane pobrane z serwisu Library.WebApi. Aplikacja dostępna jest pod adresem *localhost* na porcie 90 wewnątrz kontenera.
- Library.WebApi – serwis, który wystawia dwa endpointy, jeden odpowiedzialny za zwracanie listy książek w bibliotece, drugi który pozwala wypożyczyć książkę o zadanym Id.
- Library.NotificationService2 – aplikacja, która wyświetla w konsoli informacje o wypożyczeniu danego egzemplarza. Aplikacja jest zasubskrybowana do wiadomości o wypożyczeniu, którą nadaje serwis Library.WebApi poprzez kolejkę RabbitMQ.

Każdy z projektów wymaga przekazania zmiennych konfiguracyjnych.

- Library.WebApi potrzebuje adresu serwera RabbitMq oraz danych potrzebnych do autoryzacji do komunikacji kolejkami,
- Library.NotificationService2 potrzebuje adresu serwera RabbitMq oraz danych potrzebnych do autoryzacji do komunikacji kolejkami,
- Library.Web potrzebuje znać adres serwisu Library.WebApi, żeby wykonywać zapytania do niego.

Aplikacje napisane w .NET Core, które wykorzystują klasę *HostBuilder* (wszystkie w solucji opierają się na niej) pozwalają na załadowanie konfiguracji na wiele różnych sposobów. Na potrzeby laboratorium zostaną omówione dwa z nich:

- Zmienne środowiskowe – aplikacja w momencie uruchomienia czytuje wszystkie zmienne środowiskowe ustawione w środowisku uruchomieniowym i załaduje je do swojej pamięci
- Plik *appsettings.json* – aplikacja szuka w katalogu, w którym jest uruchamiana, pliku *appsettings.json*, a potem załaduje jego zawartość do pamięci.

Tak załadowane dane konfiguracyjne można zmapować na obiekty klas, dzięki czemu w przejrzysty sposób można korzystać z nich w aplikacji.

UWAGA! – aplikacje wczytują konfigurację obiema metodami, najpierw wczytując plik *appsettings.json*, a potem zmienne środowiskowe. Jeśli ustawisz zmienną obiema metodami, zostanie zapisana wartość z zmiennej środowiskowej!

Wszystkie aplikacje znajdujące się w solucji obsługują obie metody ładowania zmiennych konfiguracyjnych. W każdym projekcie można znaleźć plik *appsettings.json*, w którym wystarczy tylko podmienić potrzebne wartości zmiennych, aby aplikacja zaczęła działać.

5. Zadania laboratoryjne

0. Uruchomienie solucji Library na komputerze lokalnym.

- a. Pobrać ze strony prowadzącego pliki do laboratorium z Docker'a cz.1, rozpakować i uruchomić solucję. W celu jednoczesnego uruchomienia usług, należy we właściwościach solucji włączyć opcję **Wiele projektów start-upowych** i ustawić akcję wymaganych projektów na **Uruchom**. Kluczowe jest, aby w kolumnie **Debuguj element** jawnie zmienić domyślny *IIS Express* na profil projektu (np. *http* lub

- c. Domyślnie solucja skonfigurowana jest do współpracy z serwerem RabbitMQ dostępnym na localhost'cie (dwa pliki appsettings.json w Library.WebApi i Library.NotificationService2):

```
"RabbitMq": {  
  "Username": "guest",  
  "Password": "guest",  
  "ServerAddress": "rabbitmq://localhost"  
},
```

Jeśli RabbitMQ nie jest zainstalowany na localhost proszę wpisać własne dane uwierzytelniające do konta z platformy: <https://www.cloudamqp.com/>. Jeśli w konsoli pokaże się wiadomość o takiej treści:

```
RabbitMQ Connect Failed: Broker unreachable: guest@localhost:5672/  
RabbitMQ Connect Failed: Broker unreachable: guest@localhost:5672/
```

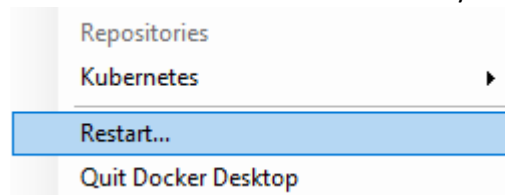
To znaczy, że prawdopodobnie adres/dane logowania są błędne. Po pomyślnej konfiguracji i prawidłowym zalogowaniu możemy wypożyczyć książkę odpytując odpowiedni endpoint w przeglądarce: http://localhost:<NUMER_PORTU>/api/library/rent/<NUMER_KSIĄŻKI>. W konsoli aplikacji Library.NotificationService2 powinniśmy dostać stosowny komunikat (rys.3):

```
The Miecz Przeznaczenia with id: 1 was rented  
dbug: MassTransit.Messages[0]  
RECEIVE rabbitmq://localhost/notification-service dc0e0000-cd7e-708b-17aa-08dec085ec41 Library.WebApi.Models.BookR  
ented Library.NotificationService2.RentedBookConsumer(00:00:00.0139884)
```

Rysunek 3: Informacja aplikacji Library.NotificationService2 o poprawnym wypożyczeniu książki.

1. Ściągnięcie i uruchomienie kontenera RabbitMQ.

- a. Działający lokalnie system będziemy krok po kroku przenosić do kornerów Dockera. W pierwszym kroku uruchomimy obraz serwera RabbitMQ. Proszę pobrać obraz i uruchomić kontener z RabbitMQ.
- i. Obraz nazywa się **rabbitmq:management** i znajduje się na oficjalnym dockerhubie. W przypadku z problemów z komunikacją z serwerem Dockera przydatny może okazać się jego restart. Dla Docker Desktop wybieramy opcję Restart z menu Dockera z zasobnika systemowego:



- ii. Kontener powinien mieć ustawione następujące mapowania portów:
"5672:5672",
"15672:15672"
- b. Zmodyfikować dane konfiguracyjne (pliki appsettings.json) w aplikacji Library.WebApi oraz Library.NotificationService2 do komunikacji w obrazem:

```
"RabbitMq": {  
  "Username": "guest",  
  "Password": "guest",  
  "ServerAddress": "rabbitmq://localhost"  
},
```

- c. Uruchomić całą sołucję lokalnie, tak, aby aplikacje Library.WebApi i Library.NotificationService2 połączyły się z RabbitMQ działającym w kontenerze z zadania nr. 3.
- d. Ponownie proszę odpytać odpowiedni endpoint w przeglądarce: http://localhost:<NUMER_PORTU>/api/library/rent/<NUMER_KSIĄŻKI> i pokazać, że aplikacja Library.NotificationService2 odebrała komunikat o wypożyczeniu książki (rys.4).

2. Zbudowanie obrazu zawierającego aplikację Library.NotificationService2 (bez użycia Dockerfile).

- a. W kolejnym kroku zbudujemy obraz zawierający aplikację Library.NotificationService2. Proszę uruchomić nowe okno wiersza poleceń albo powershella, ewentualnie opuścić kontener RabbitMQ bez zamykania go. Następnie, proszę pobrać obraz `mcr.microsoft.com/dotnet/sdk:8.0` i uruchomić kontener o nazwie `notificationsservice` z mapowaniem portów „92:8080” (adres_hosta:adres_wewnętrzny_kontenera).
- b. Proszę dostosować konfigurację w pliku appsettings.json do komunikacji w obrazem `rabbitmq:management` (szczegóły będą opisane w pkt. 4g):

```
"RabbitMq": {  
  "Username": "guest",  
  "Password": "guest",  
  "ServerAddress": "rabbitmq://rabbit"  
},
```

- c. Skopiować do kontenera `notificationsservice` zawartość folderu `Library.NotificationService2` (zawarty w plikach potrzebnych do laboratorium).
- d. Podłączyć się do uruchomionego kontenera z poziomu wiersza poleceń lub konsoli powershell.
- e. Uruchomić komendę w kontenerze: `dotnet restore`.
- f. Uruchomić komendę w kontenerze: `dotnet publish -c Release -o out`.
- g. Wejść do katalogu `out` i uruchomić komendę w kontenerze: `dotnet Library.NotificationService2.dll`.

Wynikiem powinien być podobny komunikat w kontenerze:

```
Hosting environment: Production  
Content root path: /app  
Now listening on: http://[::]:80  
Application started. Press Ctrl+C to shut down.
```

- h. Sprawdzić czy serwis jest dostępny pod adresem hosta: <http://localhost:92>:

Notification Service started!

- i. Wyjść z kontenera.
- j. Utrwalić aktualny stan kontenera jako obraz `notificationsserviceimg`.
- k. Sprawdzić, że znajduje się on na liście dostępnych obrazów.
- l. Uruchomić obraz tak aby uruchomiona została również aplikacja znajdująca się wewnątrz kontenera.
- m. Ponownie sprawdzić czy serwis jest dostępny pod adresem hosta: <http://localhost:92>.

3. Połączenie kontenera RabbitMQ z kontenerem Library.NotificationService2 przy pomocy docker network.

- Ponieważ dwa kontenery nie widzą siebie nawzajem poprzez maszynę hosta. W takim przypadku trzeba je przypiąć do tej samej sieci. Służą do tego następujące komendy:
- `docker network create my-network` – utworzenie prostej sieci o nazwie *my-network* pozwalającej na komunikację pomiędzy kontenerami uruchamianymi na jednej maszynie Dockera.
`docker network connect my-network <ID_KONTENERA_RABBITA> --alias rabbit` – podpięcie kontenera (w tym przypadku *rabbitmq*) do sieci.
`docker run ... --network my-network` – uruchomienie kontenera wraz z podpięciem do sieci.
- Komendą `docker inspect <NAZWA_SIECI>` proszę sprawdzić które kontenery udało się przypiąć do sieci.
- Komendą `docker inspect <ID_KONTENERA>` proszę czy kontenery posiadają poprawne aliasy.
- Uruchomić tylko dwie aplikacje lokalnie (Library.WebApi i Library.Web), bez Library.NotificationService2 tak, żeby połączyły się z RabbitMQ działającym w kontenerze z zadania nr. 1.
- Ponownie proszę odpytać odpowiedni endpoint w przeglądarce: http://localhost:<NUMER_PORTU>/api/library/rent/<NUMER_KSIAŻKI> i pokazać, że aplikacja Library.NotificationService2 odebrała komunikat o wypożyczeniu książki (rys.4).

Wynikiem powinien być podobny komunikat w kontenerze:

```
RabbitMQ Connect Failed: Broker unreachable: guest@rabbit:5672/
RabbitMQ Connect Failed: Broker unreachable: guest@rabbit:5672/
Hosting environment: Production
Content root path: /app/out
Now listening on: http://[::]:8080
Application started. Press Ctrl+C to shut down.
The Malowany Czlowiek. Ksiega 1 with id: 2 was rented
The Mikolajek with id: 5 was rented
```

Błędy: Problemy z połączeniem z rabbitmq może wystąpić podczas stosowania osobnych komend:

...

```
docker run -d --name CONTAINER_NAME -p PORT:PORT IMAGE_NAME
docker network connect NETWORK_NAME CONTAINER_NAME --alias ALIAS_NAME
```

...

Należy zastosować pojedynczą komendę:

...

```
docker run -d --name CONTAINER_NAME -p PORT:PORT --network NETWORK_NAME
IMAGE_NAME
```

4. Zbudowanie obrazu zawierającego aplikację Library.WebApi z wykorzystaniem Dockerfile.

- W kolejnym kroku zbudujemy obraz zawierający aplikację Library.WebApi ale z wykorzystaniem Dockerfile. Proszę dodać do projektu Library.WebApi plik Dockerfile.
- Proszę dostosować konfigurację w pliku appsettings.json do komunikacji w obrazem *rabbitmq:management*.
- Zbudować i uruchomić kontener z aplikacją Library.WebApi.

- d. Aplikacja powinna być dostępna z przeglądarki na porcie 91 (Powinna się pokazać po wpisaniu adresu: <http://localhost:91/api/library/rented/>). Należy dodać mapowanie portów przy uruchamianiu kontenera z portu **91** na maszynie hosta na port **8080** w kontenerze.
- e. Przypiąć kontener do tej samej sieci pamiętając o ponownym nadaniu aliasu pod którym będzie widoczny kontener *rabbitmq:management*.
- f. Ponownie proszę odpytać odpowiedni endpoint w przeglądarce: http://localhost:<NUMER_PORTU>/api/library/rent/<NUMER_KSIAŻKI> i pokazać, że aplikacja Library.NotificationService2 odebrała komunikat o wypożyczeniu książki (rys.4).
- g. Uruchomić lokalnie aplikację Library.Web.
- h. Aplikacja powinna mieć zmodyfikowany plik appsettings.json tak, żeby połączyła się z kolejką RabbitMQ z kontenerem z obrazu *rabbitmq:management*.
- i. Wejść na stronę <http://localhost:90> i pokazać wynik (rys. 3).

5. Zaliczenie laboratorium.

- a. Proszę przesłać następujące pliki:
 - i. print screen'y dokumentujące wykonanie każdego z podpunktów laboratorium
 - ii. historię z każdego okna poleceń. W linuxie służy do tego komenda: `history` > `d:/history.txt`, a w Windows (cmd): `doskey /HISTORY` > `d:\history.txt`, (powershell): `Get-History` | `Export-CSV D:\history.csv`
 - iii. wszystkie pliki Dockerfile
 - iv. skrypty – jeśli były wykorzystywane
 - v. pliki appsettings.json wykorzystywane do komunikacji z obrazem *rabbitmq:management*.

6. Błędy i pomoc w rozwiązywaniu problemów (Troubleshooting)

1. Brak połączenia z aplikacją w przeglądarce (Błąd: *Site cannot be reached / Connection Refused*)

- **Przyczyna:** Od wersji .NET 8 Microsoft zmienił domyślny port wewnątrz kontenerów (w obrazach *aspnet:8.0*) z **portu 80 na port 8080**. Jeśli kontener nasłuchuje w środku na porcie 8080, a Ty zmapowałeś port hosta na port 80 (np. `-p 91:80`), żądania trafią w próżnię.
- **Rozwiązanie:**
 - Zmień mapowanie portów podczas uruchamiania kontenera na standard .NET 8, kierując ruch na port 8080 wewnątrz kontenera, np.:

```
docker run -d --name library-api -p 91:8080 --network my-network library-api-img
```

- **Alternatywnie** możesz wymusić, aby aplikacja w kontenerze działała na starym porcie 80, przekazując zmienną środowiskową w komendzie ``docker run``:

```
docker run -d -e ASPNETCORE_HTTP_PORTS=80 -p 91:80 ...
```

2. Komunikat: *RabbitMQ Connect Failed: Broker unreachable*

- **Przyczyna 1:** Kontener z aplikacją .NET próbuje łączyć się przez adres `localhost`. Pamiętaj, że dla kontenera `localhost` oznacza... sam ten kontener, a nie Twoją maszynę fizyczną.
- **Rozwiązanie:** W pliku `appsettings.json` kontenerowej aplikacji zmień adres serwera na alias sieciowy nadany kontenerowi RabbitMQ (np. `"ServerAddress": "rabbitmq://rabbit"`).
- **Przyczyna 2:** Kontenery nie zostały prawidłowo podpięte do tej samej sieci wirtualnej.
- **Rozwiązanie:** Upewnij się, że oba kontenery pracują w sieci `my-network`. Sprawdź to komendą:

```
docker network inspect my-network
```

W sekcji `Containers` musisz widzieć zarówno kontener RabbitMQ, jak i kontener usługi powiadomień/WebAPI.

3. Kontener wyłącza się natychmiast po starcie (Błąd: *OCI runtime create failed*)

- **Przyczyna:** Błąd składniowy w instrukcji `ENTRYPOINT` w pliku `Dockerfile`, polegający na braku spacji pomiędzy słowem kluczowym a nawiasem kwadratowym (np. `ENTRYPOINT["dotnet", ...]`).
- **Rozwiązanie:** Docker nie potrafi zinterpretować takiego polecenia. Dodaj spację po słowie kluczowym:

```
ENTRYPOINT ["dotnet", "Library.Web.dll"]
```

4. Problem z blokowaniem portów na hoście (Błąd: `port is already allocated`)

- **Przyczyna:** Próba uruchomienia kontenera na porcie (np. 90, 91, 5672), który jest już zajęty przez lokalnie uruchomioną instancję Visual Studio (proces ``dotnet run``) lub inną usługę.
- **Rozwiązanie:** Przed uruchomieniem kontenerów zamknij wszystkie działające w tle konsole z uruchomionymi lokalnie projektami .NET (`Ctrl + C`` w oknie wiersza poleceń).